

Principles of Programming Languages

Lecture 3: Abstract syntax and semantics of expressions

Andrei Arusoaie¹

¹Department of Computer Science

October 19, 2020

Outline

Alphabet. Lexical analysis. Parsing.

Parse Trees

Abstract syntax trees

Sentences in a programming language

Which phrases are correct?

- ▶ `int x; x = x + 2`
- ▶ `int x; x = x + 2;`
- ▶ `if (a > 0) then x = 1; else x = -1;`
- ▶ `(a > 0) ? x = 1 : x = -1;`

Sentences in a programming language

Which phrases are correct?

- ▶ `int x; x = x + 2`
- ▶ `int x; x = x + 2;`
- ▶ `if (a > 0) then x = 1; else x = -1;`
- ▶ `(a > 0) ? x = 1 : x = -1;`

Sentences in a programming language

Which phrases are correct?

- ▶ `int x; x = x + 2`
- ▶ `int x; x = x + 2;`
- ▶ `if (a > 0) then x = 1; else x = -1;`
- ▶ `(a > 0) ? x = 1 : x = -1;`

Sentences in a programming language

Which phrases are correct?

- ▶ `int x; x = x + 2`
- ▶ `int x; x = x + 2;`
- ▶ `if (a > 0) then x = 1; else x = -1;`
- ▶ `(a > 0) ? x = 1 : x = -1;`

Overview: alphabet, lexical analysis, syntax.

- ▶ ***Alphabet*** : set of (allowed) symbols
- ▶ *Lexical analysis*: identify the sequence of symbols constituting the *words* (or *tokens*)
 - ▶ *Lexical rules*
- ▶ *Syntax*: describes which sequences of words constitute “legal” phrases
 - ▶ *Grammars*

Overview: alphabet, lexical analysis, syntax.

- ▶ *Alphabet* : set of (allowed) symbols
- ▶ *Lexical analysis*: identify the sequence of symbols constituting the *words* (or *tokens*)
 - ▶ *Lexical rules*
- ▶ *Syntax*: describes which sequences of words constitute “legal” phrases
 - ▶ *Grammars*

Overview: alphabet, lexical analysis, syntax.

- ▶ *Alphabet* : set of (allowed) symbols
- ▶ *Lexical analysis*: identify the sequence of symbols constituting the *words* (or *tokens*)
 - ▶ *Lexical rules*
- ▶ *Syntax*: describes which sequences of words constitute “legal” phrases
 - ▶ *Grammars*

Alphabet

The Alphabet of C from the Standard has 96 symbols:

- ▶ a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, z
- ▶ A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z
- ▶ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- ▶ ! " # % & ' () * + , - . /
- ▶ : ; < = > ? [\] ^ _ { | } ~
- ▶ *Separators*: space, horizontal and vertical tab, form feed, newline

Lexical analysis

Problem: Given a sequence of characters, find the pieces with assigned meaning from that sequence: words or *tokens*

Example:

- ▶ Input: `if (a > 0) then x = 1; else x = -1;`
- ▶ Output: `if, (, a, >, 0,), then, x, =, 1,;, else, x, =, -1,;`
- ▶ *Tokens = pieces with assigned/identified meaning*

Lexical analyzer (lexer) = a program that implements an algorithm that solves the problem above

Lexical analysis

Problem: Given a sequence of characters, find the pieces with assigned meaning from that sequence: words or *tokens*

Example:

- ▶ Input: `if (a > 0) then x = 1; else x = -1;`
- ▶ Output: `if, (, a, >, 0,) , then, x, =, 1,, ;, else, x, =, -1,, ;`
- ▶ *Tokens = pieces with assigned/identified meaning*

Lexical analyzer (lexer) = a program that implements an algorithm that solves the problem above

Lexical analysis

Problem: Given a sequence of characters, find the pieces with assigned meaning from that sequence: words or *tokens*

Example:

- ▶ Input: `if (a > 0) then x = 1; else x = -1;`
- ▶ Output: `if, (, a, >, 0,) , then, x, =, 1,, ;, else, x, =, -1,, ;`
- ▶ *Tokens = pieces with assigned/identified meaning*

Lexical analyzer (lexer) = a program that implements an algorithm that solves the problem above

Lexical analysis

Problem: Given a sequence of characters, find the pieces with assigned meaning from that sequence: words or *tokens*

Example:

- ▶ Input: `if (a > 0) then x = 1; else x = -1;`
- ▶ Output: `if, (, a, >, 0,) , then, x, =, 1, ;, else, x, =, -1, ;`
- ▶ *Tokens = pieces with assigned/identified meaning*

Lexical analyzer (lexer) = a program that implements an algorithm that solves the problem above

Example I - Lexical rules

- ▶ **Integers: 6, 0, -2, +3**
- ▶ The *alphabet* $A = \{+, -\} \cup \mathbb{N}$
- ▶ *Lexical rules*: used to describe atomic language constructions: numbers, identifiers, ...
- ▶ *Lexical rules* are expressed using *regular grammars* (see LFAC course)
- ▶ **Regular expressions**, a.k.a **regex**
 - ▶ Regex for integers: `[\+-]? \d+`

Example I - Lexical rules

- ▶ Integers: 6, 0, -2, +3
- ▶ The *alphabet* $A = \{+, -\} \cup \mathbb{N}$
- ▶ *Lexical rules*: used to describe atomic language constructions: numbers, identifiers, ...
- ▶ *Lexical rules* are expressed using *regular grammars* (see LFAC course)
- ▶ **Regular expressions**, a.k.a **regex**
 - ▶ Regex for integers: `[\+-]? \d+`

Example I - Lexical rules

- ▶ Integers: 6, 0, -2, +3
- ▶ The *alphabet* $A = \{+, -\} \cup \mathbb{N}$
- ▶ *Lexical rules*: used to describe atomic language constructions: numbers, identifiers, ...
- ▶ *Lexical rules* are expressed using *regular grammars* (see LFAC course)
- ▶ **Regular expressions**, a.k.a **regex**
 - ▶ Regex for integers: `[\+-]?\d+`

Example I - Lexical rules

- ▶ Integers: 6, 0, -2, +3
- ▶ The *alphabet* $A = \{+, -\} \cup \mathbb{N}$
- ▶ *Lexical rules*: used to describe atomic language constructions: numbers, identifiers, ...
- ▶ *Lexical rules* are expressed using *regular grammars* (see LFAC course)
- ▶ Regular expressions, a.k.a *regex*
 - ▶ Regex for integers: `[\+-]?\d+`

Example I - Lexical rules

- ▶ Integers: 6, 0, -2, +3
- ▶ The *alphabet* $A = \{+, -\} \cup \mathbb{N}$
- ▶ *Lexical rules*: used to describe atomic language constructions: numbers, identifiers, ...
- ▶ *Lexical rules* are expressed using *regular grammars* (see LFAC course)
- ▶ **Regular expressions**, a.k.a **regex**
 - ▶ Regex for integers: `[\+-]?\d+`

Parsing

Problem: how to combine the tokens in (valid) sentences?

- ▶ Answer: we define the *grammar* of the language
- ▶ Grammars allow us to transform a program given as an sequence of characters into a *syntax tree*
- ▶ Parser = program which attempts to do this transformation
- ▶ Only valid programs can be parsed!

Parsing

Problem: how to combine the tokens in (valid) sentences?

- ▶ Answer: we define the *grammar* of the language
- ▶ Grammars allow us to transform a program given as an sequence of characters into a *syntax tree*
- ▶ Parser = program which attempts to do this transformation
- ▶ Only valid programs can be parsed!

Parsing

Problem: how to combine the tokens in (valid) sentences?

- ▶ Answer: we define the *grammar* of the language
- ▶ Grammars allow us to transform a program given as an sequence of characters into a *syntax tree*
- ▶ Parser = program which attempts to do this transformation
- ▶ Only valid programs can be parsed!

Parsing

Problem: how to combine the tokens in (valid) sentences?

- ▶ Answer: we define the *grammar* of the language
- ▶ Grammars allow us to transform a program given as an sequence of characters into a *syntax tree*
- ▶ Parser = program which attempts to do this transformation
- ▶ Only valid programs can be parsed!

Parsing

Problem: how to combine the tokens in (valid) sentences?

- ▶ Answer: we define the *grammar* of the language
- ▶ Grammars allow us to transform a program given as an sequence of characters into a *syntax tree*
- ▶ Parser = program which attempts to do this transformation
- ▶ Only valid programs can be parsed!

Example II - Grammar

- ▶ Language of palindromic strings using symbols a and b
- ▶ The *alphabet* $A = \{a, b\}$
- ▶ Can we describe palindromes using regex?

Example II - Grammar

- ▶ Language of palindromic strings using symbols a and b
- ▶ The *alphabet* $A = \{a, b\}$
- ▶ Can we describe palindromes using regex?

Example II - Grammar

- ▶ Language of palindromic strings using symbols a and b
- ▶ The *alphabet* $A = \{a, b\}$
- ▶ Can we describe palindromes using regex?

Example II - Grammar

- ▶ How do we “formally” describe palindromic strings?
 - ▶ Note that there is a simple recursion of a palindromic string
 - ▶ Base: a and b are palindromic strings
 - ▶ Recursion: if s is a palindromic string then so are asa and bsb
- ▶ Examples: “aba”, “aabaa”, “bab”, etc
- ▶ *Problem?* yes: “aa”, “abba”.
- ▶ Fix: add the empty string to base, hereafter denoted by ϵ

Example II - Grammar

- ▶ How do we “formally” describe palindromic strings?
 - ▶ Note that there is a simple recursion of a palindromic string
 - ▶ Base: a and b are palindromic strings
 - ▶ Recursion: if s is a palindromic string then so are asa and bsb
 - ▶ Examples: “aba”, “aabaa”, “bab”, etc
 - ▶ Problem? yes: “aa”, “abba”.
 - ▶ Fix: add the empty string to base, hereafter denoted by ϵ

Example II - Grammar

- ▶ How do we “formally” describe palindromic strings?
 - ▶ Note that there is a simple recursion of a palindromic string
 - ▶ Base: a and b are palindromic strings
 - ▶ Recursion: if s is a palindromic string then so are asa and bsb
- ▶ Examples: “aba”, “aabaa”, “bab”, etc
- ▶ *Problem?* yes: “aa”, “abba”.
- ▶ Fix: add the empty string to base, hereafter denoted by ϵ

Example II - Grammar

- ▶ How do we “formally” describe palindromic strings?
 - ▶ Note that there is a simple recursion of a palindromic string
 - ▶ Base: a and b are palindromic strings
 - ▶ Recursion: if s is a palindromic string then so are asa and bsb
- ▶ Examples: “aba”, “aabaa”, “bab”, etc
- ▶ *Problem?* yes: “aa”, “abba”.
- ▶ Fix: add the empty string to base, hereafter denoted by ϵ

Example II - Grammar

- ▶ How do we “formally” describe palindromic strings?
 - ▶ Note that there is a simple recursion of a palindromic string
 - ▶ Base: a and b are palindromic strings
 - ▶ Recursion: if s is a palindromic string then so are asa and bsb
- ▶ Examples: “aba”, “aabaa”, “bab”, etc
- ▶ *Problem?* yes: “aa”, “abba”.
- ▶ Fix: add the empty string to base, hereafter denoted by ϵ

Example II - Grammar

- ▶ How do we “formally” describe palindromic strings?
 - ▶ Note that there is a simple recursion of a palindromic string
 - ▶ Base: a and b are palindromic strings
 - ▶ Recursion: if s is a palindromic string then so are asa and bsb
- ▶ Examples: “aba”, “aabaa”, “bab”, etc
- ▶ *Problem?* yes: “aa”, “abba”.
- ▶ Fix: add the empty string to base, hereafter denoted by ϵ

Example II - Grammar

- ▶ How do we “formally” describe palindromic strings?
 - ▶ Note that there is a simple recursion of a palindromic string
 - ▶ Base: a and b are palindromic strings
 - ▶ Recursion: if s is a palindromic string then so are asa and bsb
- ▶ Examples: “aba”, “aabaa”, “bab”, etc
- ▶ *Problem?* yes: “aa”, “abba”.
- ▶ Fix: add the empty string to base, hereafter denoted by ϵ

Example II - Grammar

- ▶ **Base case:**

- ▶ $P \rightarrow \epsilon$

- ▶ $P \rightarrow a$

- ▶ $P \rightarrow b$

- ▶ **Recursion:**

- ▶ $P \rightarrow aPa$

- ▶ $P \rightarrow bPb$

- ▶ **Context-free grammar** (you study this in your compiler course!)

Example II - Grammar

- ▶ **Base case:**

- ▶ $P \rightarrow \epsilon$

- ▶ $P \rightarrow a$

- ▶ $P \rightarrow b$

- ▶ **Recursion:**

- ▶ $P \rightarrow aPa$

- ▶ $P \rightarrow bPb$

- ▶ **Context-free grammar** (you study this in your compiler course!)

Example II - Grammar

- ▶ **Base case:**

- ▶ $P \rightarrow \epsilon$

- ▶ $P \rightarrow a$

- ▶ $P \rightarrow b$

- ▶ **Recursion:**

- ▶ $P \rightarrow aPa$

- ▶ $P \rightarrow bPb$

- ▶ **Context-free grammar** (you study this in your compiler course!)

Example II - Grammar

- ▶ Base case:

- ▶ $P \rightarrow \epsilon$

- ▶ $P \rightarrow a$

- ▶ $P \rightarrow b$

- ▶ Recursion:

- ▶ $P \rightarrow aPa$

- ▶ $P \rightarrow bPb$

- ▶ Context-free grammar (you study this in your compiler course!)

Example II - Grammar

- ▶ Base case:

- ▶ $P \rightarrow \epsilon$

- ▶ $P \rightarrow a$

- ▶ $P \rightarrow b$

- ▶ Recursion:

- ▶ $P \rightarrow aPa$

- ▶ $P \rightarrow bPb$

- ▶ Context-free grammar (you study this in your compiler course!)

Example II - Grammar

- ▶ Base case:
 - ▶ $P \rightarrow \epsilon$
 - ▶ $P \rightarrow a$
 - ▶ $P \rightarrow b$
- ▶ Recursion:
 - ▶ $P \rightarrow aPa$
 - ▶ $P \rightarrow bPb$
- ▶ Context-free grammar (you study this in your compiler course!)

Example II - Grammar

- ▶ Base case:
 - ▶ $P \rightarrow \epsilon$
 - ▶ $P \rightarrow a$
 - ▶ $P \rightarrow b$
- ▶ Recursion:
 - ▶ $P \rightarrow aPa$
 - ▶ $P \rightarrow bPb$
- ▶ Context-free grammar (you study this in your compiler course!)

Example II - Grammar

- ▶ Base case:
 - ▶ $P \rightarrow \epsilon$
 - ▶ $P \rightarrow a$
 - ▶ $P \rightarrow b$
- ▶ Recursion:
 - ▶ $P \rightarrow aPa$
 - ▶ $P \rightarrow bPb$
- ▶ **Context-free grammar** (you study this in your compiler course!)

Backus-Naur Form (BNF)

- ▶ *Meta-language* introduced by Backus and Naur to define ALGOL60
- ▶ Vocabulary:
 - ▶ **Terminals** : simple language strings; typically: *tokens* or symbols
 - ▶ **Non-terminals**: complex language constructions
- ▶ How BNF rules look like:
 - ▶ `Bool ::= "true" | "false"`
(we can use lexical rules to define "basic" non-terminals)
 - ▶ `Exp ::= Int | Exp "+" Exp | ...`

Backus-Naur Form (BNF)

- ▶ *Meta-language* introduced by Backus and Naur to define ALGOL60
- ▶ Vocabulary:
 - ▶ **Terminals** : simple language strings; typically: *tokens* or symbols
 - ▶ **Non-terminals**: complex language constructions
- ▶ How BNF rules look like:
 - ▶ `Bool ::= "true" | "false"`
(we can use lexical rules to define “basic” non-terminals)
 - ▶ `Exp ::= Int | Exp "+" Exp | ...`

Backus-Naur Form (BNF)

- ▶ *Meta-language* introduced by Backus and Naur to define ALGOL60
- ▶ Vocabulary:
 - ▶ **Terminals** : simple language strings; typically: *tokens* or symbols
 - ▶ **Non-terminals**: complex language constructions
- ▶ How BNF rules look like:
 - ▶ `Bool ::= "true" | "false"`
(we can use lexical rules to define "basic" non-terminals)
 - ▶ `Exp ::= Int | Exp "+" Exp | ...`

Backus-Naur Form (BNF)

- ▶ *Meta-language* introduced by Backus and Naur to define ALGOL60
- ▶ Vocabulary:
 - ▶ **Terminals** : simple language strings; typically: *tokens* or symbols
 - ▶ **Non-terminals**: complex language constructions
- ▶ How BNF rules look like:
 - ▶ `Bool ::= "true" | "false"`
(we can use lexical rules to define “basic” non-terminals)
 - ▶ `Exp ::= Int | Exp "+" Exp | ...`

Backus-Naur Form (BNF)

- ▶ *Meta-language* introduced by Backus and Naur to define ALGOL60
- ▶ Vocabulary:
 - ▶ **Terminals** : simple language strings; typically: *tokens* or symbols
 - ▶ **Non-terminals**: complex language constructions
- ▶ How BNF rules look like:
 - ▶ `Bool ::= "true" | "false"`
(we can use lexical rules to define “basic” non-terminals)
 - ▶ `Exp ::= Int | Exp "+" Exp | ...`

BNF - example

► Simple expressions language:

```
Int   ::=  [\+-]?[0-9]+  
Exp   ::=  Int  
        |  Exp "+" Exp  
        |  "(" Exp ")"
```


Derivations

Derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Exp + Int)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (Int + 6)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Grammar

Int	::=	$[\backslash+-]?[0-9]^+$	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

Derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Exp + Int)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (Int + 6)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Grammar

Int	::=	$[\backslash+-]?[0-9]^+$	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		" (" Exp ")" "	(e_3)

Derivations

Derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Exp + Int)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (Int + 6)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Grammar

Int	::=	$[\backslash+-]?[0-9]^+$	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

Derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Exp + Int)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (Int + 6)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Grammar

Int	::=	$[\backslash+-]?[0-9]^+$	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

Derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Exp + Int)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (Int + 6)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Grammar

Int	::=	$[\backslash+-]?[0-9]^+$	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

Derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Exp + Int)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (Int + 6)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Grammar

Int	::=	$[\backslash+-]?[0-9]^+$	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

Derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Exp + Int)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (Int + 6)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Grammar

Int	::=	$[\backslash+-]?[0-9]^+$	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		" (" Exp ")" "	(e_3)

Derivations

Derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Exp + Int)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (Int + 6)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Grammar

Int	::=	$[\backslash+-]?[0-9]^+$	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		" (" Exp ")" "	(e_3)

Derivations

Derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Exp + Int)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (Int + 6)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Grammar

Int	::=	$[\backslash+-]?[0-9]^+$	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

Derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Exp + Int)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (Int + 6)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Grammar

Int	::=	$[\backslash+-]?[0-9]^+$	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		" (" Exp ")" "	(e_3)

Derivations

Another possible derivation

Exp $\rightarrow (e_2)$
Exp + Exp $\rightarrow (e_3)$
Exp + (Exp) $\rightarrow (e_2)$
Exp + (Exp + Exp) $\rightarrow (e_1)$
Exp + (Int + Exp) $\rightarrow (e_1)$
Exp + (Int + Int) $\rightarrow (e_1)$
Int + (Int + Int) $\rightarrow (i_1)$
Int + (4 + Int) $\rightarrow (i_1)$
Int + (4 + 6) $\rightarrow (i_1)$
2 + (4 + 6)

Int ::= [\+-]?[0-9]+ (i_1)
Exp ::= Int (e_1)
| Exp "+" Exp (e_2)
| "(" Exp ")" (e_3)

Derivations

Another possible derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Int + Exp)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (4 + Int)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Int	::=	<code>[\+-]?[0-9]+</code>	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

Another possible derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Int + Exp)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (4 + Int)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Int	::=	<code>[\+-]?[0-9]+</code>	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

Another possible derivation

Exp	$\rightarrow$	(e_2)
Exp + Exp	$\rightarrow$	(e_3)
Exp + (Exp)	$\rightarrow$	(e_2)
Exp + (Exp + Exp)	$\rightarrow$	(e_1)
Exp + (Int + Exp)	$\rightarrow$	(e_1)
Exp + (Int + Int)	$\rightarrow$	(e_1)
Int + (Int + Int)	$\rightarrow$	(i_1)
Int + (4 + Int)	$\rightarrow$	(i_1)
Int + (4 + 6)	$\rightarrow$	(i_1)
2 + (4 + 6)		

Int	::=	<code>[\+-]?[0-9]+</code>	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

Another possible derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Int + Exp)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (4 + Int)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Int	::=	$[\backslash+-]?[0-9]^+$	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

Another possible derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Int + Exp)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (4 + Int)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Int	::=	<code>[\+-]?[0-9]+</code>	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

Another possible derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Int + Exp)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (4 + Int)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Int	::=	<code>[\+-]?[0-9]+</code>	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

Another possible derivation

Exp	$\rightarrow(e_2)$
Exp + Exp	$\rightarrow(e_3)$
Exp + (Exp)	$\rightarrow(e_2)$
Exp + (Exp + Exp)	$\rightarrow(e_1)$
Exp + (Int + Exp)	$\rightarrow(e_1)$
Exp + (Int + Int)	$\rightarrow(e_1)$
Int + (Int + Int)	$\rightarrow(i_1)$
Int + (4 + Int)	$\rightarrow(i_1)$
Int + (4 + 6)	$\rightarrow(i_1)$
2 + (4 + 6)	

Int	::=	<code>[\+-]?[0-9]+</code>	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

Another possible derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Int + Exp)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (4 + Int)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Int	::=	$[\backslash+-]?[0-9]^+$	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

Derivations

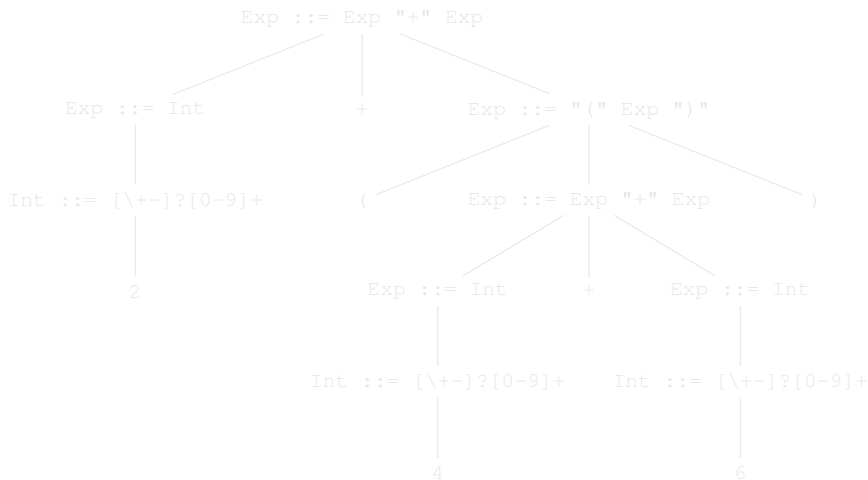
Another possible derivation

Exp	$\rightarrow (e_2)$
Exp + Exp	$\rightarrow (e_3)$
Exp + (Exp)	$\rightarrow (e_2)$
Exp + (Exp + Exp)	$\rightarrow (e_1)$
Exp + (Int + Exp)	$\rightarrow (e_1)$
Exp + (Int + Int)	$\rightarrow (e_1)$
Int + (Int + Int)	$\rightarrow (i_1)$
Int + (4 + Int)	$\rightarrow (i_1)$
Int + (4 + 6)	$\rightarrow (i_1)$
2 + (4 + 6)	

Int	::=	<code>[\+-]?[0-9]+</code>	(i_1)
Exp	::=	Int	(e_1)
		Exp "+" Exp	(e_2)
		"(" Exp ")"	(e_3)

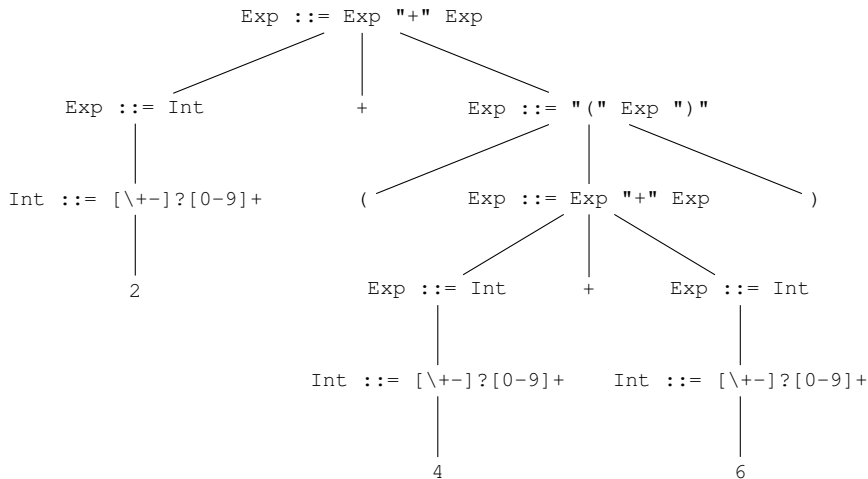
Parse trees

Parse tree for $2 + (4 + 6)$:



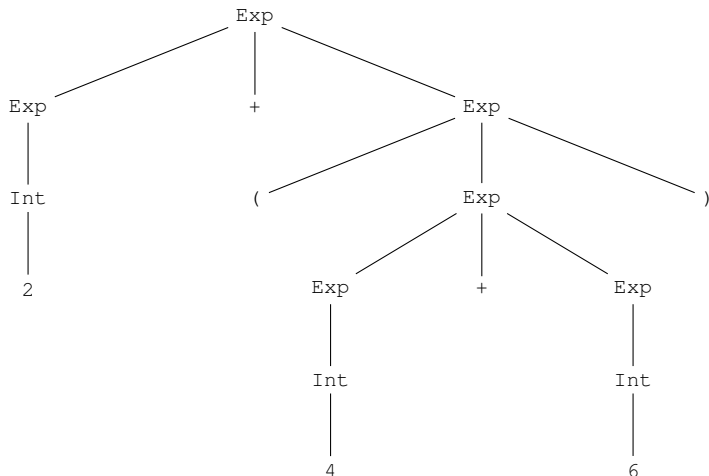
Parse trees

Parse tree for $2 + (4 + 6)$:



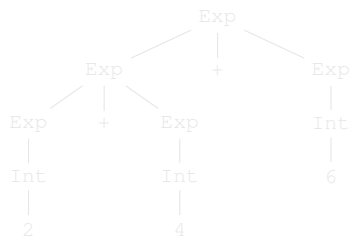
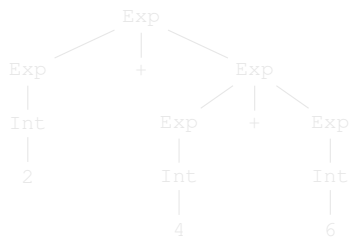
Parse trees: simplified

Parse tree for $2 + (4 + 6)$:



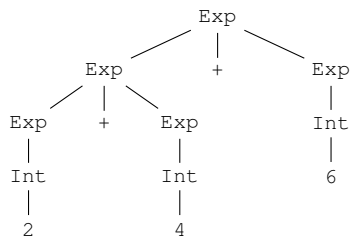
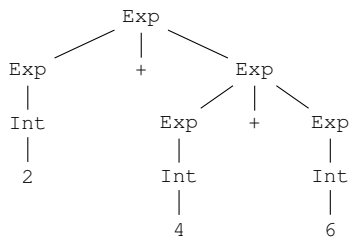
Multiple parses available

Possible parse trees for $2 + 4 + 6$:



Multiple parses available

Possible parse trees for $2 + 4 + 6$:



Ambiguities

Possible parse trees for $2 + 4 + 6$:

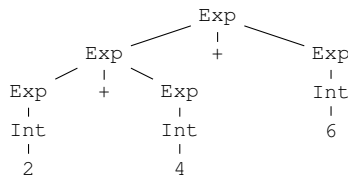
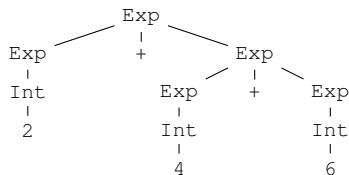


► Solutions?

- Use the parentheses defined in the syntax: ' (' and ') '
- Encode some kind of associativity: **left** or **right**

Ambiguities

Possible parse trees for $2 + 4 + 6$:

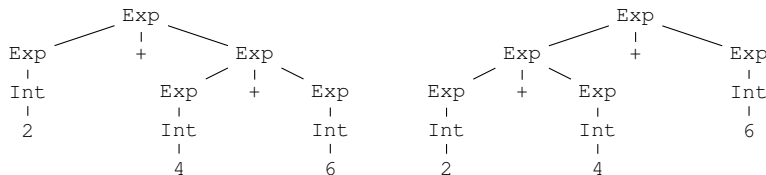


► Solutions?

- Use the parentheses defined in the syntax: ' (' and ') '
- Encode some kind of associativity: **left** or **right**

Ambiguities

Possible parse trees for $2 + 4 + 6$:

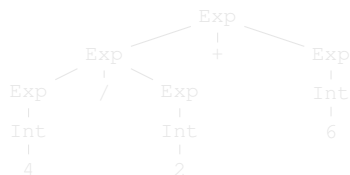


► Solutions?

- Use the parentheses defined in the syntax: ‘(’ and ‘)’
- Encode some kind of associativity: **left** or **right**

Priorities

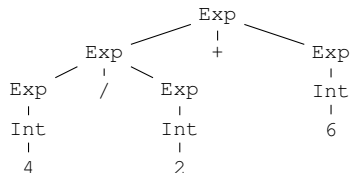
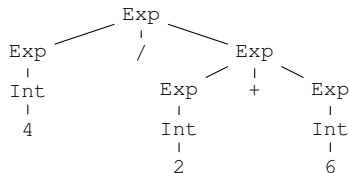
Possible parse trees for $4 / 2 + 6$:



► What parse tree is correct in this case?

Priorities

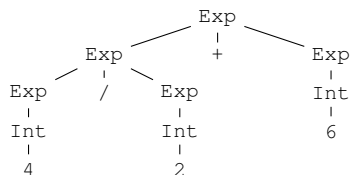
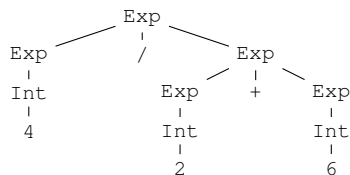
Possible parse trees for $4 / 2 + 6$:



► What parse tree is correct in this case?

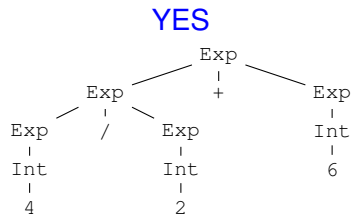
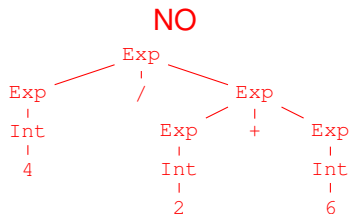
Priorities

Possible parse trees for $4 / 2 + 6$:



- What parse tree is correct in this case?

Priorities



Abstract Syntax Trees

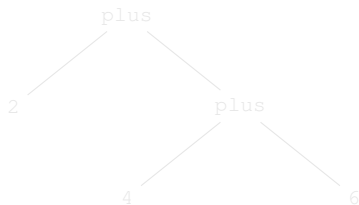
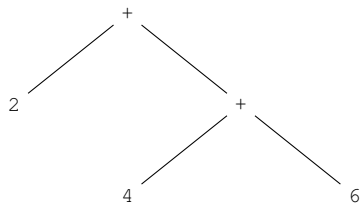
- ▶ An AST is a *tree* representation of the structure of a program
- ▶ "Abstract" = not every detail occurring in the text of the program is present in the AST representation
- ▶ Compilers use ASTs as the main data structure

Abstract Syntax Trees

- ▶ An AST is a *tree* representation of the structure of a program
- ▶ "Abstract" = not every detail occurring in the text of the program is present in the AST representation
- ▶ Compilers use ASTs as the main data structure

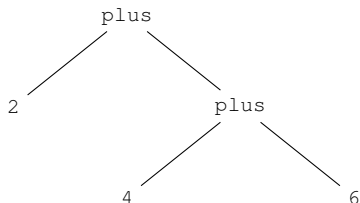
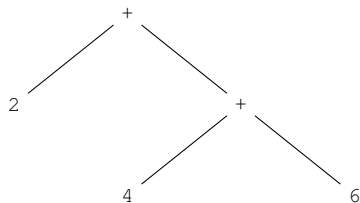
Example: arithmetic expressions

AST for $2 + (4 + 6)$:



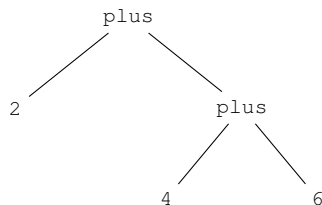
Example: arithmetic expressions

AST for $2 + (4 + 6)$:



Example: arithmetic expressions

AST for $2 + (4 + 6)$:



```
Inductive Exp :=  
| number : nat -> Exp  
| plus : Exp -> Exp -> Exp.
```

```
Coercion number : nat >-> Exp.
```

```
Check (plus 2 (plus 4 6)).  
plus 2 (plus 4 6)  
      : Exp
```

Demo

- ▶ Arithmetic expressions
- ▶ Boolean Expressions

Bibliography

- ▶ Sections 2.1-2.4 from *Programming Languages: Principles and Paradigms*, Maurizio Gabbrielli, Simone Martini; 2010.
Link: http://websrv.dthu.edu.vn/attachments/newsevents/content2415/Programming_Languages_-_Principles_and_Paradigms_thereds1106.pdf